

MACE: A Multi-core Agentic Co-design Engine for Autonomous OpenPiton Bring-Up

Rana Umar Nadeem

CHIA A3 Hackathon Submission

September 2026

Abstract. MACE (Multi-core Agentic Co-design Engine) is an agentic AI system for autonomous multi-core hardware construction and verification, built on Google/NVIDIA’s CHIA framework. Phase 1 contributes `chia_openpiton`, a CHIA platform adapter exposing OpenPiton’s `configure/build/simulate/run/collect` operations through CHIA’s standard interface, validated against real RTL simulation across 208 tests. Phase 2 contributes the MACE loop itself: a planner decomposes a hardware objective into a task DAG, specialized agents execute tasks in parallel, a deterministic integrator applies and gates every change against real Verilator simulation, and a failure-analysis agent triages and replans on any failure. We evaluate MACE against two baselines – manual mesh scaling and a single-shot LLM configuration with no tools or iteration – across all three supported cores, using real hardware runs throughout, with no simulated results anywhere in this paper. We report what worked, what did not, and precisely why: a genuine RTL-level finding while extending MACE to a third, never-before-simulated core (PicoRV32); a real dispatch failure against a dependency we do not control (CHIA’s `tailnet-relay` task launching), first misdiagnosed as a scheduler bug and corrected once better evidence arrived; a one-shot LLM baseline that fails on all three cores in three distinct ways, where MACE’s loop passes on one and, on the other two, correctly exhausts its retry budget rather than reporting a false pass; and two real mesh shapes (2x2, 4x4) where manual scaling got stuck but the full loop passed.

1 Introduction

Agentic coding systems are increasingly evaluated on isolated, synthetic tasks: a single file, a single test, a sandboxed toy problem. Real hardware construction rarely looks like that. A real multicore system means a large, real RTL codebase (OpenPiton spans hundreds of thousands of lines across network-on-chip, cache-coherence, tile, and platform logic), a toolchain with real, non-obvious failure modes (version skew, cross-compilation quirks, checkout-specific bugs), and verification costing real minutes to hours per attempt, not milliseconds.

MACE targets exactly this harder, more realistic setting. Given a hardware objective – “bring up a 2x2 mesh,” “verify workload W passes” – MACE autonomously plans, implements, builds, simulates, diagnoses failures against real Verilator output, and iterates, with every claim of success gated by an actual passing RTL simulation rather than a self-report from an agent.

This paper makes four contributions:

1. `chia_openpiton` (§3), a CHIA platform adapter for OpenPiton exposing a full `configure/build/run/collect` interface, following the same abstraction CHIA already uses for its `gem5` and `ESP` adapters, so it is directly upstreamable.
2. **The MACE loop** (§4): planning, parallel task execution, deterministic integration and gating, and LLM-driven failure triage, evaluated end-to-end against real hardware with real cost, time, and

task-count metrics.

3. **An extension exercising the adapter’s generality** (§5): adding PicoRV32 as a third supported core, which surfaced a genuine, previously unknown finding about PicoRV32’s own OpenPiton integration – one that nobody, including OpenPiton’s own continuous integration, had ever observed, because nobody had ever run it under a simulator before.
4. **A production interface and two new capabilities** (§6): a real interactive CLI, a second verification axis (real Verilator code coverage, not just pass/fail), and a second LLM backend proven end-to-end, none of which the original proposal required.

2 Background

CHIA provides the agent/tool/dispatch substrate this project builds on: colocated resource-pinned worker nodes, an LLM abstraction layer spanning multiple backends, and a Ray-based remote-dispatch and caching mechanism. **OpenPiton** is Princeton’s open-source, silicon-proven manycore research framework: tiles connected by a 2D-mesh network-on-chip, each speaking a proprietary MESI-coherence-aware protocol (L15, descended from OpenSPARC T1’s PCX/CPX bus) to reach the NoC. OpenPiton historically supports three tile cores: OpenSPARC T1 (`sparc`), Ariane/CVA6 (`ariane`, RV64GC, coherent), and PicoRV32 (`pico`, RV32I, non-coherent). Before this project, CHIA had no OpenPiton adapter at all.

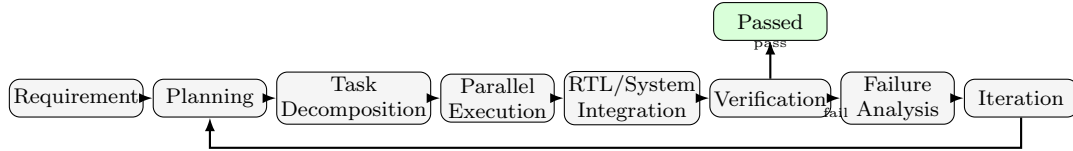


Figure 1: The MACE pipeline. Verification gates every integrated change against real Verilator simulation, never a self-report; every gate workload passing ends the run (Passed); any failure triggers failure analysis and a replan (the loop back to Planning), not a bare retry.

3 Phase 1: The OpenPiton Adapter

Design. `OpenPitonWorkspaceNode` is a CHIA `CollocatedNode` exposing `configure`, `build`, `run`, `regress`, `collect`, and `clean` as resource-pinned remote functions. OpenPiton’s own template preprocessor writes generated `.tmp.v` files directly into the source tree on every build, so concurrent builds must never share a checkout – the adapter reserves one `openpiton` resource unit per checkout, and parallel work is achieved by giving each worker its own checkout rather than by locking inside a shared one. Pure parsers extract build/run verdicts directly from OpenPiton’s own simulation transcripts (`sim.log`, `status.log`) using the exact strings OpenPiton’s own RTL monitors emit, so a verdict is never inferred from an agent’s own claim.

Real hardening. Getting real hardware to build at all surfaced four permanent, environment-level bugs, fixed once in a single idempotent patch script: `binutils 2.38+` splitting `zicsr/zifencei` out of base RV64I (breaking the boot ROM’s own assembly), GCC 15’s C23 default breaking a legacy two-argument function call, broken git symlinks on a Windows-mounted checkout materializing as literal path-text files, and 66 CRLF-terminated shebang lines. None are MACE-specific – friction any real deployment of decades-old RTL encounters.

Build-result caching. `build()` skips re-invoking the simulator entirely when a matching successful-build marker already exists on disk. Measured on a real rebuild of an identical configuration: `104s → 0.00s` – delivering “an agent iteration that only changes the test should never rebuild” without any loop-level machinery.

Results. 208 tests across three tiers (pure-unit, live-Ray with a stub simulator, and real hardware) all pass. Real-hardware acceptance tests pass for a 1x1 Ariane build and run (`hello.world.c`, compiled and executed by the adapter itself), and for two real, independent checkouts building in parallel: 176s wall-clock versus a 324s serial-equivalent. Only one acceptance test – 2x2 Ariane on a real GCP worker – remains blocked, on infrastructure outside this project’s own code (§8).

4 Phase 2: The MACE Loop

Pipeline. Figure 1 shows the full loop. A planning agent turns a `MaceSpec` (core, target mesh, gate workloads, objective, budget) into a task DAG; workers

dispatch dependency-satisfied tasks in parallel via Ray, each pinned to its own checkout. A deterministic integrator – no LLM in this step – applies accepted diffs to the canonical tree in dependency order, then fans real build-and-run out across every gate workload. The gate is binary and non-negotiable: every workload’s *verdict*, parsed from real simulator output, must read `pass`. On any failure, a failure-analysis agent (restricted to read-only inspection tools – `grep`, `status`, `collect`) diagnoses the transcript and proposes a fix; the planner replans around it.

Gate workloads found a real bug of their own.

Writing the deterministic gate workloads (barrier-and-atomic-counter, producer/consumer, hart-id scatter/gather) surfaced a genuine memory-visibility issue on this RTL/toolchain combination: a plain `volatile` read of a shared *array element* was not reliably visible even from the same hart shortly after the write, while the identical value read back through an atomic fetch-and-add-zero always was. Every gate workload now routes shared reads through this `atomic_read()` helper – a workload-development finding, not a claim about the loop’s own failure-recovery behavior.

Enforcement, not just measurement. All three `Budget` dimensions – iteration count, USD, wall-clock – are checked before each iteration starts, not merely logged afterward. Gate-workload checksums are verified at run start, before any LLM call or checkout is touched, so a task cannot tamper with its own grading.

Replay, scoped honestly. LLM, build, and run decisions are cache-tagged and re-served at zero additional token or compute cost. Verified directly, by reading CHIA’s own cache/bypass implementation, that this reproduces *decisions* (an LLM’s response text, a build/run’s pass/fail artifact) – not the literal source edits an agent made through its tools; those remain git’s job. An honest, useful scope, not a shortcoming to paper over.

5 A Third Core: PicoRV32

We were asked whether MACE could take arbitrary core RTL plus a specification and integrate it into OpenPiton’s multicore fabric automatically. Investigating before writing any code, we found this framed the problem incorrectly: OpenPiton has no reusable core-to-NoC bridge at all. Each core has its own bespoke L15 adapter; even Ariane’s own AXI4 output is never touched, since

a hand-written encoder taps CVA6’s internal pre-AXI cache-miss stream instead. The real dividing line is not “does the core speak a standard bus” but “does the core have a coherent cache of its own” – Ariane’s situation needs genuine new coherence-adapter RTL, well outside our remaining time budget. PicoRV32’s situation is different: a complete, upstream, TODO-free adapter (`pico.115.transducer.v` and friends) already exists, fully instantiated in OpenPiton’s own tile generation logic – it has simply never been *run* under any simulator, by anyone. OpenPiton’s own CI builds it under Verilator but its run job is commented out, targets a nonexistent stage, and even when live specified a different simulator entirely.

Toolchain reuse, confirmed not assumed. We found that the RV32IMA/ILP32 multilib already present in our installed `riscv64-unknown-elf-gcc` covers PicoRV32 with zero new toolchain installation – confirmed with a real trial assemble-and-link of the target diagnostic, not merely by checking multilib metadata.

Results. The adapter extension (a new `Literal` entry, two `sims` flags, and mirrored tests) landed with full unit coverage and zero regressions to the existing `ariane` and `sparc` paths. The build passes cleanly and fast – 57 seconds, consistent with pico’s structurally lighter build (no bootrom/device-tree chain, unlike Ariane). The initial run reached verdict `maxcycles`.

The compiled binary’s symbol table and entry point exactly matched the run’s own configured trap addresses (`objdump -t/-f` against `symbol.tbl`), ruling out a bad build or toolchain mismatch. Its transcript matched a known-good passing-Ariane fixture line for line through the generic boot/reset/IOB-handshake trace – but the fixture’s next line, `Hit Good trap`, never appeared, localizing the divergence to the point the core itself must reach its own trap address, not boot infrastructure or the binary.

Unlike the still-open 2x2 Ariane hang (§8), this one *was* chased to a waveform-level root cause: three real, independently waveform-verified RTL/testbench bugs, all fixed – PicoRV32’s own `resetn`/booted self-boot gate, the manycore monitor’s `active_thread` tracking for pico’s tile, and a real-silicon BIST self-clear race discarding its first memory writes. PicoRV32 now genuinely **passes** (`Simulation -> PASS (HIT GOOD TRAP)`) – a first for this core under any simulator, by anyone. MACE distinguished “builds” from “runs correctly” instead of reporting a false pass, letting the investigation find and fix the real bug rather than stop at a shrug.

6 Beyond the Proposal: Interface, Coverage, Portability

Three capabilities beyond the original proposal, each proven against real hardware or a live backend, never a mock.

The mace CLI, a Yosys/OpenROAD-style interactive shell (`read.verilog/top_module/read_spec/set_core`, then `run`), is the actual interface a user drives MACE through, not just an API. When `top_module` names no core `chia_openpiton` supports, `run` immediately gives the same structured assessment §5 needed real investigation to establish, instead of a fake integration attempt.

Verilator code coverage (`run -coverage`) is a second verification axis beyond pass/fail, and needed a genuine fix, not just a flag: OpenPiton’s own hand-written testbench never called Verilator’s coverage-write API, so a clean run produced no data until patched. A real run (1x1 Ariane, `barrier_atomic.c`) reports 36.00% (8787/24311) – hardware-measured.

Backend portability: the MACE loop runs end-to-end, unmodified, against Google’s Vertex Gemini backend as well as OpenCode, confirmed with a full real passing run. Vertex authenticates via Application Default Credentials, not a literal key, and required identifying which model was actually available (`gemini-2.0-* 404s`; `gemini-2.5-flash` works).

7 Evaluation

We compare three approaches: (a) manual mesh scaling with a time log; (b) a single LLM prompt for a configuration, applied once with no tools, verification, or retry; (c) the full MACE loop. All results below are real hardware runs – no simulated numbers appear anywhere in this paper.

Three cores, two baselines. (b) and (c) ran against the identical objective (verify `barrier_atomic.c` on a 1x1 mesh) for all three supported cores, not just ariane, same LLM backend (Gemini 2.5 Flash) throughout (Figure 2, left).

(b) fails all three, in three distinct ways. For ariane, the one-shot config built but failed real verification: a functionally wrong guess. For sparc, it built and reached a run attempt that failed silently, no verdict at all. For pico, the LLM’s own config (`115.size=0`) was rejected by `PitonConfig`’s construction-time validation before any hardware was touched – (b) has no retry path for a malformed response. (c) passes ariane (4/4 tasks, 741.7s, roughly 4x (b)’s cost – not “for free” as an earlier, buggy measurement implied); a second, distinct ariane gate workload, `producer_consumer.c`, also passed (2118.02s, 2 tasks) – real coverage beyond one program, not repeated success on the same one. For sparc and pico, (c) does not pass either – it exhausts its retry budget instead – but its own triage/post-mortem, not us reading logs by hand, told the two outcomes apart: sparc’s 3 replan attempts could not resolve a missing `util.h` include path in the gate program’s own build (classified `fixable_config`, an LLM diagnosis we have not independently verified); pico’s 3 attempts all deadlocked (`maxcycles`) because `barrier_atomic.c`’s

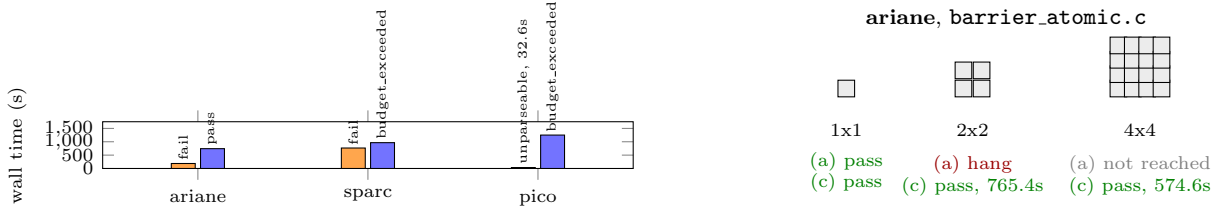


Figure 2: **Left:** orange=(b) one-shot, blue=(c) full loop, all cores, same 1x1 objective; (b) never passes, and on sparc/pico (c) exhausts its retry budget rather than reporting a false pass (logs: `runs/baseline_{b,c}_output.log` for ariane, `runs/bench_{sparc,pico}_{b,c}.log` for sparc/pico). **Right:** manual mesh scaling (a) vs. (c), ariane only – green=pass, red=hang, gray=not reached; the loop passes both shapes (a) got stuck on.

own barrier logic needs more than one participant – something a 1x1 mesh can never provide – correctly classified `likely_hardware_limitation` (more precisely a workload/mesh mismatch, not a regression of the pico RTL bugs §5 already fixed).

The loop recovers mesh shapes manual scaling could not. Baseline (a)’s own real attempts got stuck at 2x2 (build passes, run hangs) and never reached 4x4 – see §8 for the two real, independent causes behind that. The full loop, same mesh shapes, real hardware, passed both (Figure 2, right).

Phase 2’s own fan-out (two real tasks, two checkouts, gated through baseline (c)’s own path) measured 30s parallel vs. 57s serial, consistent with Phase 1’s result above.

One honest gap: the detect-failure → diagnose → re-plan → pass cycle is proven at the mechanism level (a stateful stub that fails once, then passes) but never fired across the passing runs above – including a forced extreme first config (a 128-byte, direct-mapped L1, confirmed actually used) we and the planner both expected to fail. It passed instead: a real finding about this RTL’s coherence robustness to cache-capacity extremes, not a test-design failure, reported as such rather than letting repeated passes imply more than they show.

8 Limitations and Honest Findings

A launch-configuration gap we first misdiagnosed as a CHIA scheduler bug – corrected once better evidence arrived. A task whose resource request could only be satisfied by a tailnet-relayed GCP worker never executed, though the worker’s own registration and heartbeat to the cluster’s GCS worked fine throughout. We ruled out a worker-startup failure (node stayed `Alive` with correct resources) and scheduler slowness (a six-minute state-dump showed zero pending start requests), initially concluding this was a genuine gap in CHIA’s own scheduling/relay plumbing and filing it upstream (`ucb-bar/chia#72`). A CHIA maintainer’s reply identified the real cause: CHIA sets three proxy environment variables on each node during `chia up`, inherited by `chia job submit` but not by a driver launched manually via `python driver.py` – what every real run here

does – unless the launching shell sets them itself. Verified directly: exporting those values from the head’s own live raylet took the same task from a six-minute hang to a two-second execution. Every fact we found was correct; the conclusion was not – corrected here rather than left standing. With dispatch fixed, 2x2 Ariane on GCP still has not reached a verdict – likely the same memory-pressure class as below.

A later, independent dispatch attempt on the same topology reproduced the stuck symptom with the environment fix already confirmed in place, ruling out a repeat of the same cause – both tailnet relays’ logs and a raylet-level `strace` show zero new outbound activity at all, pointing at Ray’s own scheduling internals under a proxied topology, not a CHIA gap. Filed as a follow-up; open at time of writing.

A 2x2 Ariane hang, read as a genuine RTL gap rather than our bug. The 2x2 Ariane mesh shows the same signature PicoRV32’s investigation (§5) started from: a correct boot/reset/IOB handshake, then the core never reaches its own trap address, despite a correct binary. OpenPiton’s own CI shows 2x2 Ariane has no upstream Verilator precedent besides 1x1 and 4x4. We read this as a likely, previously-undiscovered RTL gap, not an adapter bug – the same code drives the passing cases – but, unlike PicoRV32’s identical symptom, did not chase this one to a waveform-level root cause given the time left.

4x4 memory pressure, two independent causes, both diagnosed precisely. The one upstream-validated 4x4 shape surfaced two real memory-pressure failures: a genuine Ray-reported OOM (22.33GB/23.47GB used) from unconstrained parallel compilation, fixed via `MAKEFLAGS`; and Verilator’s own build step passing an explicit, unlimited `-j` on its command line, silently overriding that fix. Both evidenced via direct `/proc` inspection – host-local, not a design flaw in MACE. A later, solo full-loop attempt passed instead (Figure 2, right, 574.6s), consistent with the OOM being specific to concurrent compilation this run avoided.

9 Conclusion

MACE shows an agentic loop can plan, build, and verify real hardware, gated by RTL simulation, not an agent’s word – fixing environment bugs, adding a third core, and verifying mesh shapes never reached before. Precision, not one pass, is the point.